Final Project: PTX3, CXCR2, and RUNDC3A Expression, Vitamin D, and Preeclampsia

Ethan Lin

2026-11-10

This file produces every figure and table for the final LaTeX report.

Variables of interest

- **Genes:** *PTX3* (from Submission 1), *CXCR2*, and *RUNDC3A*
- **Continuous covariate:** maternal baseline 25-hydroxyvitamin D (ng/mL)
- **Additional continuous covariates (Table 1, NEW):** maternal age (years) and BMI (kg/m^2)
- **Categorical covariate 1:** pregnancy outcome (Normal vs Preeclampsia)
- **Categorical covariate 2 (derived from continuous):** vitamin D status (Deficient < 20, Insufficient 20-29.9, Sufficient 30+ ng/mL)

```
library(tidyverse)
library(kableExtra)  # Latex summary table
library(pheatmap)    # for clustered heatmap
```

1. Load the raw files

```
# Raw files, unchanged from Canvas
covar     <- read_csv("data/GSE85307_covar-1.csv")
expr      <- read_csv("data/GSE85307_geneExpression.csv")
geneNames <- read_csv("data/GSE85307_geneNames.csv")
```

2. Clean the covariate data (same steps as Submission 1)

Each covariate value is stored as text such as `"bmi_kg/m2: 37.52"`, so keep only the part after `": "` and convert numbers to numeric.

```
covar_clean <- covar |>
  # Rename first column name by position
  rename(subjectID = 1) |>
  # For every column except the IDs, keep only the text after ": "; consulted
  # the help of Claude for this step to help with str_removal, detecting patterns
  mutate(across(-c(subjectID, geoID),
                ~ str_remove(.x, "^.*: "))) |>
  # Convert the continuous variables from text to numeric
  mutate(across(c(mat_age_yrs,
                  delivery_gestAge_wks,
                  sampleCollection_gestAge_wks,
                  mat_BMI,
                  mat_vitD),
```

```
                as.numeric)) |>
  mutate(
    # Pregnancy outcome as a factor, with normal and preeclampsia levels
    preg_preeclampsia = factor(preg_preeclampsia,
                               levels = c("Normal", "Preeclampsia")),
    # Turn asthma into a labeled factor, either 0 or 1
    mat_asthma = factor(mat_asthma,
                        levels = c("0", "1"),
                        labels = c("Healthy", "Asthma")),
    # convert to factor variable
    batch_Microarray = factor(batch_Microarray),
    # turn continuous mat_vitD to categorical vitD_status variable
    # chose the intervals and cutoffs based off of existing clinical studies
    # and standard medical practices
    vitD_status = cut(mat_vitD,
                      breaks = c(-Inf, 20, 30, Inf),
                      right  = FALSE,
                      labels = c("Deficient (<20)",
                                 "Insufficient (20-29.9)",
                                 "Sufficient (30+)"))
  )

summary(covar_clean)
count(covar_clean, preg_preeclampsia, vitD_status)
```

Sample size check for Methods section later. Final sample size and justification for removing any data points.

```
# 157 samples, no missing values, and one sample per subject,
# so no samples need to be removed
nrow(covar_clean)
```

```
## [1] 157
```

```
sum(is.na(covar_clean))
```

```
## [1] 0
```

```
n_distinct(covar_clean$subjectID)
```

```
## [1] 157
```

3. Build a long data frame with all of the genes

Building a data frame that holds every gene, because the function needs a data frame, so it can pull out whichever genes it is given.

```
expr_long <- expr |>
  # Probe ID column has no header; name it so it can be joined
  rename(PROBEID = 1) |>
  # Attach gene names from the linker file
  left_join(geneNames, by = "PROBEID") |>
  # convert to long format, so one row per gene per sample
  pivot_longer(cols = starts_with("GSM"),
               names_to  = "geoID",
               values_to = "expression")

# double check that every probe has a gene name, and no gene appears twice
```

```r
sum(is.na(expr_long$GENE))
expr_long |>
  distinct(PROBEID, GENE) |>
  count(GENE) |>
  filter(n > 1)  |>
  nrow()
```

4. Merge with covariates and verify the match

```r
# check if set of sample IDs in both files?
setequal(unique(expr_long$geoID), covar_clean$geoID)

analysis_long <- covar_clean |>
  inner_join(expr_long, by = "geoID")

# Expect 157 samples x 500 genes = 78,500 rows, with 157 unique samples
# passed the check
nrow(analysis_long)
n_distinct(analysis_long$geoID)

# Check if merged PTX3 value for one sample equals the raw file value
analysis_long |>
  filter(geoID == "GSM2264346", GENE == "PTX3")  |>
  pull(expression)

expr |>
  rename(PROBEID = 1)  |>
  filter(PROBEID == 8083594)  |>
  pull(GSM2264346)
```

5. Shared plot theme and colors (same from each previous submission)

```r
# One theme for consistency across all plots; publication quality
theme_pub <- theme_classic(base_size = 14) +
  theme(plot.title = element_text(face = "bold", hjust = 0.5),
        axis.title = element_text(face = "bold"),
        legend.position = "top",
        legend.title = element_text(face = "bold"))

# shared colors
outcome_colors <- c("Normal" = "#4C72B0", "Preeclampsia" = "#C44E52")
```

6. Plotting function (same from Submission 2)

plot_genes() function takes the following inputs: a data frame and a vector of one or more gene names. For each gene it loops through, it makes the three Submission 1 plots (histogram, scatterplot vs vitamin D, boxplot by vitamin D status and outcome) and prints them, optionally saves them, and returns them in a named list.

```r
# takes the input of a data frame, a vector of genes, into the figures directory
plot_genes <- function(df, genes, save = FALSE, out_dir = "figures") {

  # First initialize an empty list
```

```r
all_plots <- list()

# Loop through each gene provided and produce the plots (huge for loop!)
for (gene in genes) {

  # Keep only this gene's rows (one row per sample)
  # rename input data frame as gene_df
  gene_df <- df |>
    filter(GENE == gene)
  # Axis label reused by all three plots
  y_lab <- paste(gene, "expression (log2 intensity)")

  # Plot 1: histogram showing distribution of expression; essentially same
  # code as submission 1
  p_hist <- ggplot(gene_df, aes(x = expression)) +
    # 0.05-wide bins suit the narrow range of log2 expression values
    geom_histogram(binwidth = 0.05, fill = "#3A7D7C", color = "white",
                   linewidth = 0.3) +
    # Dashed line at the median as a reference point
    geom_vline(aes(xintercept = median(expression)),
               linetype = "dashed", color = "#1F2A37", linewidth = 0.6) +
    # Bars sit directly on the x-axis, with some space on top
    scale_y_continuous(expand = expansion(mult = c(0, 0.05))) +
    labs(title = paste("Distribution of", gene, "Expression"),
         x = y_lab,
         y = "Number of samples",
         caption = paste0("Dashed line = median; n = ", nrow(gene_df))) +
    theme_pub

  # Once again same thing but for plot 2: scatterplot of vitamin D
  # vs expression. same code as submission 1, just changing the inputs.
  p_scatter <- ggplot(gene_df, aes(x = mat_vitD, y = expression)) +
    # One point per woman, colored by outcome
    geom_point(aes(color = preg_preeclampsia), alpha = 0.7, size = 2.5) +
    # Overall linear trend with 95% confidence band
    geom_smooth(method = "lm", color = "black", se = TRUE) +
    scale_color_manual(values = outcome_colors) +
    labs(title = paste("Maternal Vitamin D vs", gene, "Expression"),
         x = "Maternal 25(OH)D at baseline (ng/mL)",
         y = y_lab,
         color = "Pregnancy outcome") +
    theme_pub

  # Same thing again, but for plot 3: boxplot by both categorical variables
  p_box <- ggplot(gene_df, aes(x = vitD_status, y = expression,
                               fill = preg_preeclampsia)) +
    # Boxes side by side for each outcome; hide outliers because
    # all points are drawn below
    geom_boxplot(outlier.shape = NA, alpha = 0.8,
                 position = position_dodge(width = 0.8)) +
    # Individual women, jittered sideways and aligned with their box
    geom_point(position = position_jitterdodge(jitter.width = 0.15,
                                               dodge.width  = 0.8),
```

```
                  size = 1.2, alpha = 0.6) +
      scale_fill_manual(values = outcome_colors) +
      # edit in that particular looped gene
      labs(title = paste(gene, "Expression by Vitamin D Status and Outcome"),
           x = "Maternal vitamin D status (ng/mL)",
           y = y_lab,
           fill = "Pregnancy outcome") +
      theme_pub

    # Print all three plots so they appear in the knitted document
    print(p_hist)
    print(p_scatter)
    print(p_box)

    # Save the plots named by gene if save is set to TRUE, the default input
    # is that save is set as FALSE initially; this is optional, that way doesn't
    # continuously rewrite plots each time functino is run
    if (save) {
      ggsave(file.path(out_dir, paste0("histogram_", gene, ".png")),
             p_hist, width = 7, height = 5, dpi = 300)
      ggsave(file.path(out_dir, paste0("scatter_vitD_", gene, ".png")),
             p_scatter, width = 7, height = 5, dpi = 300)
      ggsave(file.path(out_dir, paste0("boxplot_vitD_outcome_", gene, ".png")),
             p_box, width = 8, height = 5, dpi = 300)
    }

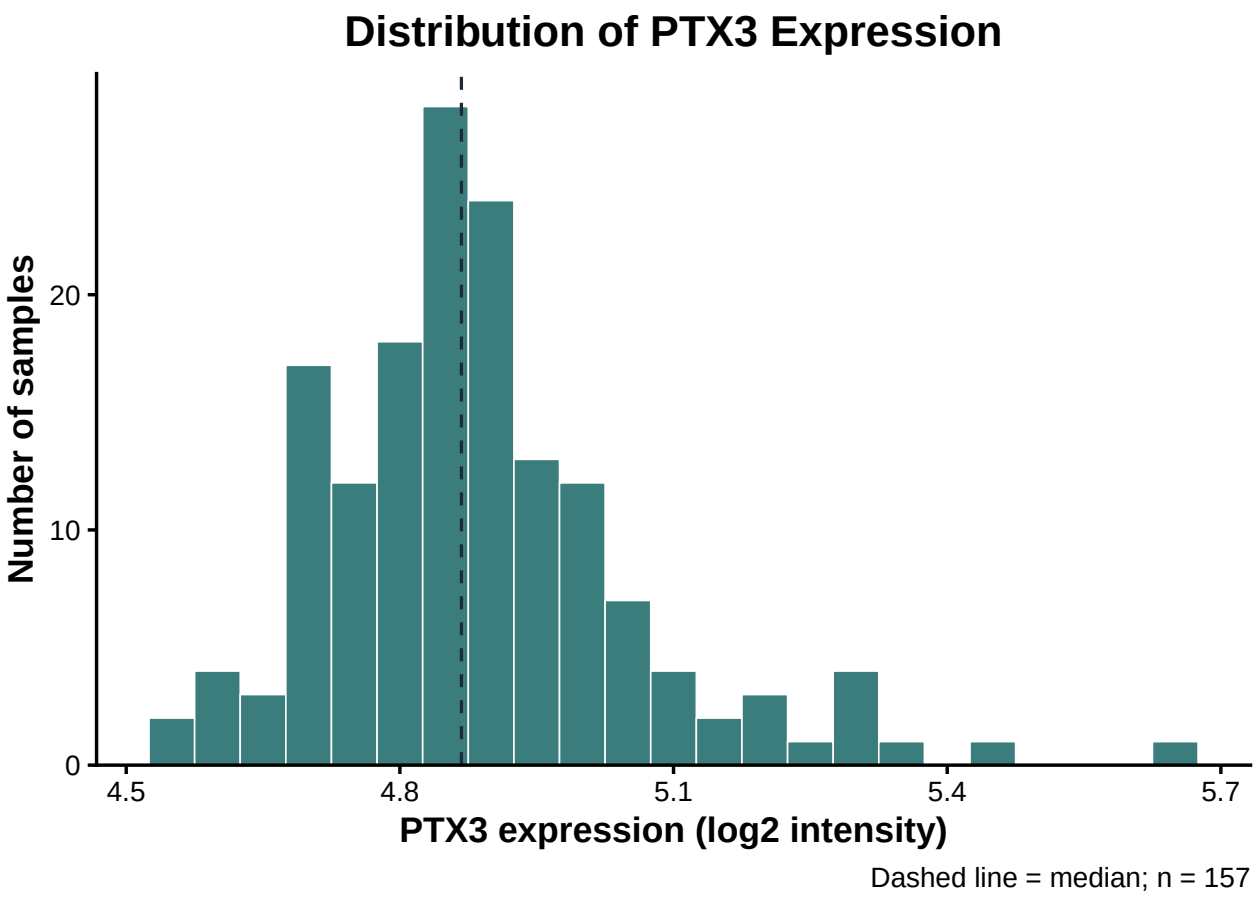
    # Store this gene's plots in the list under the gene's name
    # This way when the loop finishes you can pull out any single plot later
    # without rerunning the function
    all_plots[[gene]] <- list(histogram = p_hist,
                              scatter   = p_scatter,
                              boxplot   = p_box)
  }
}
```

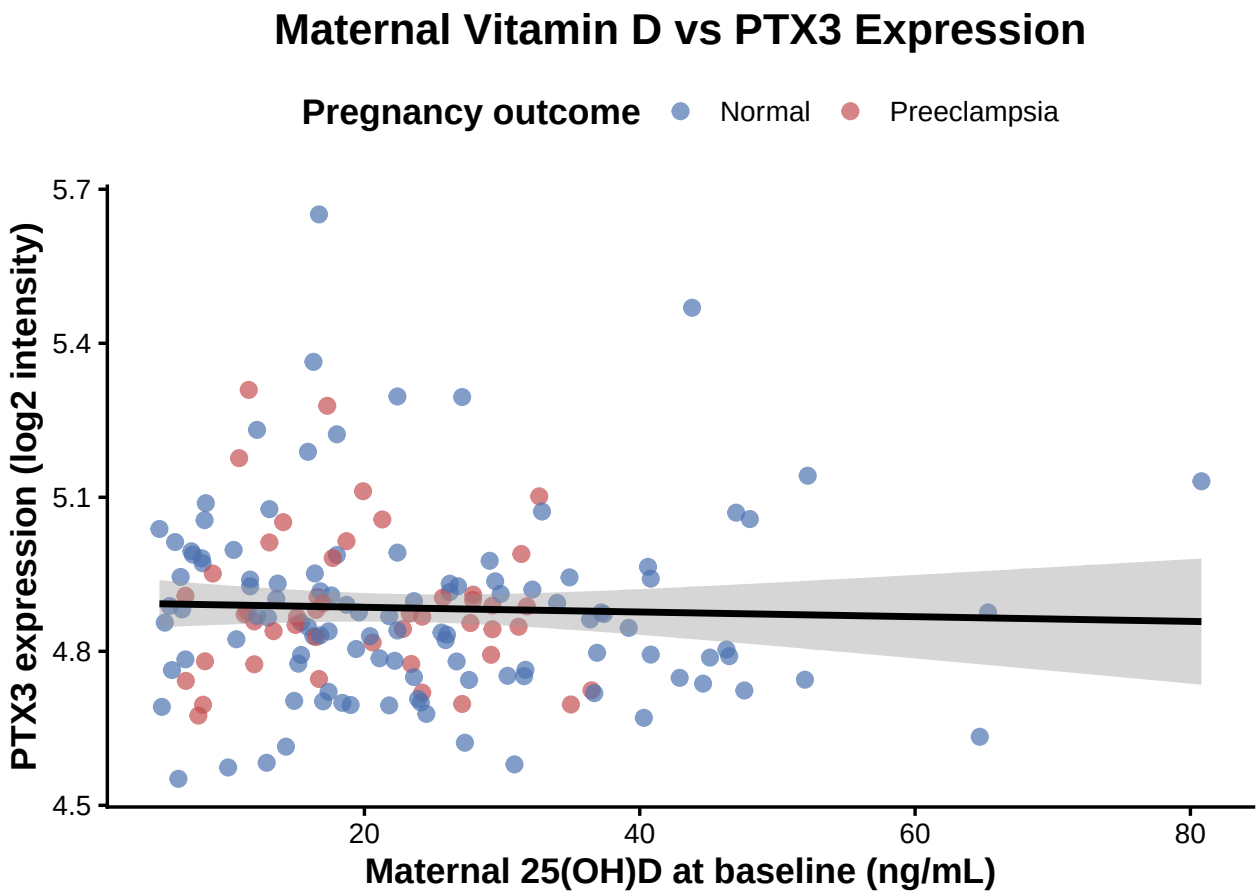
7. Run the function on three genes

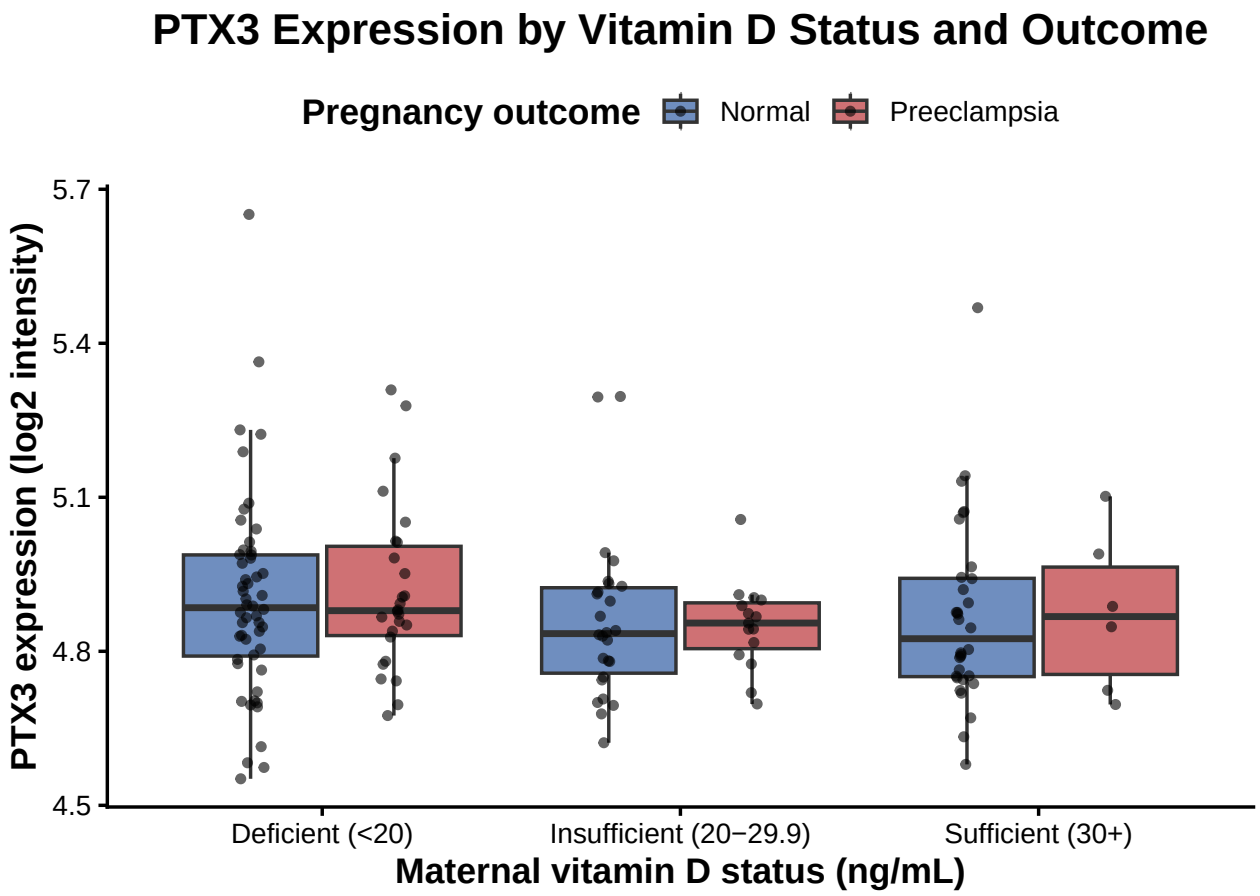
```
# interested genes in the list
genes_of_interest <- c("PTX3", "CXCR2", "RUNDC3A")

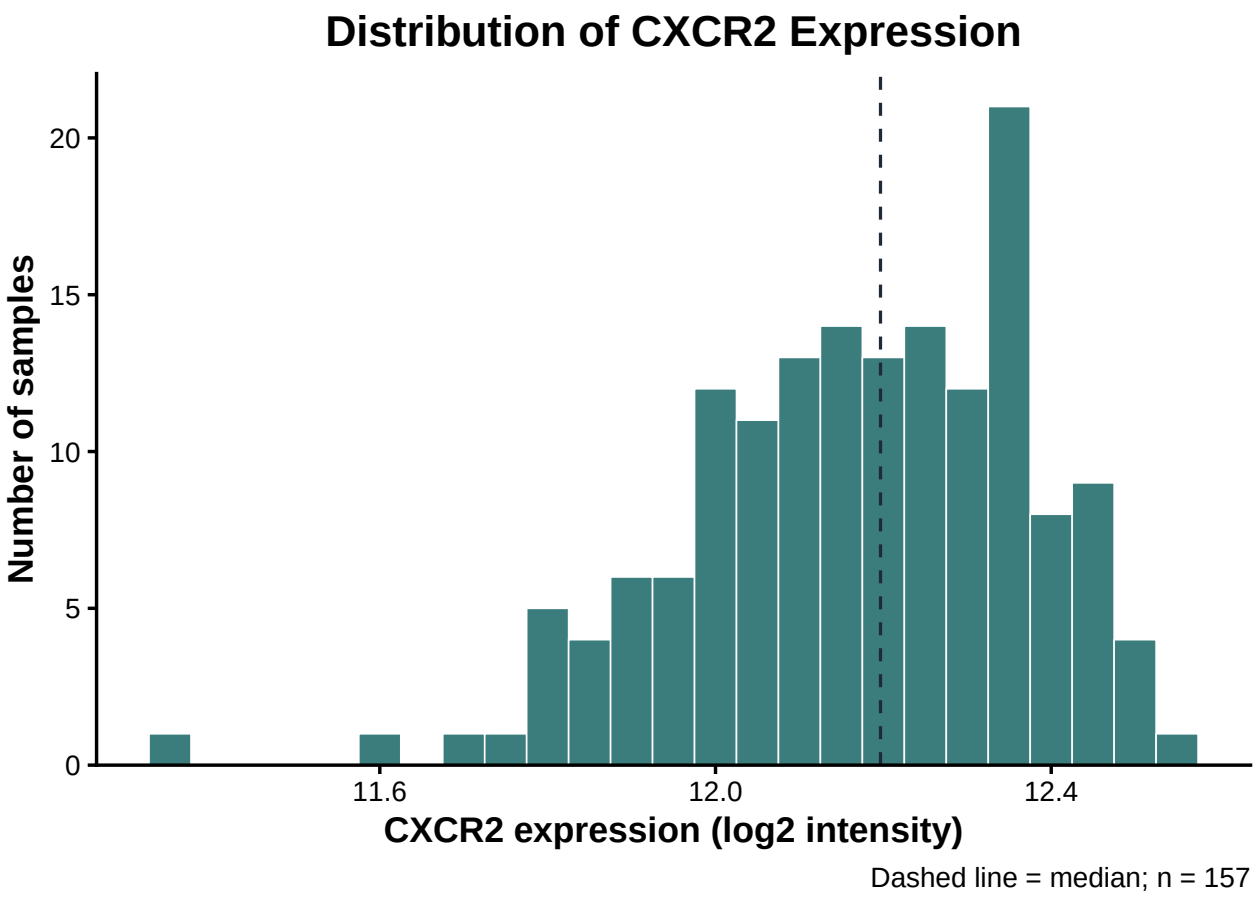
# Set save = TRUE to save the 9 figures in figures folder
# Set save = FALSE in final pdf/markdown because don't want to keep running
# this over and over

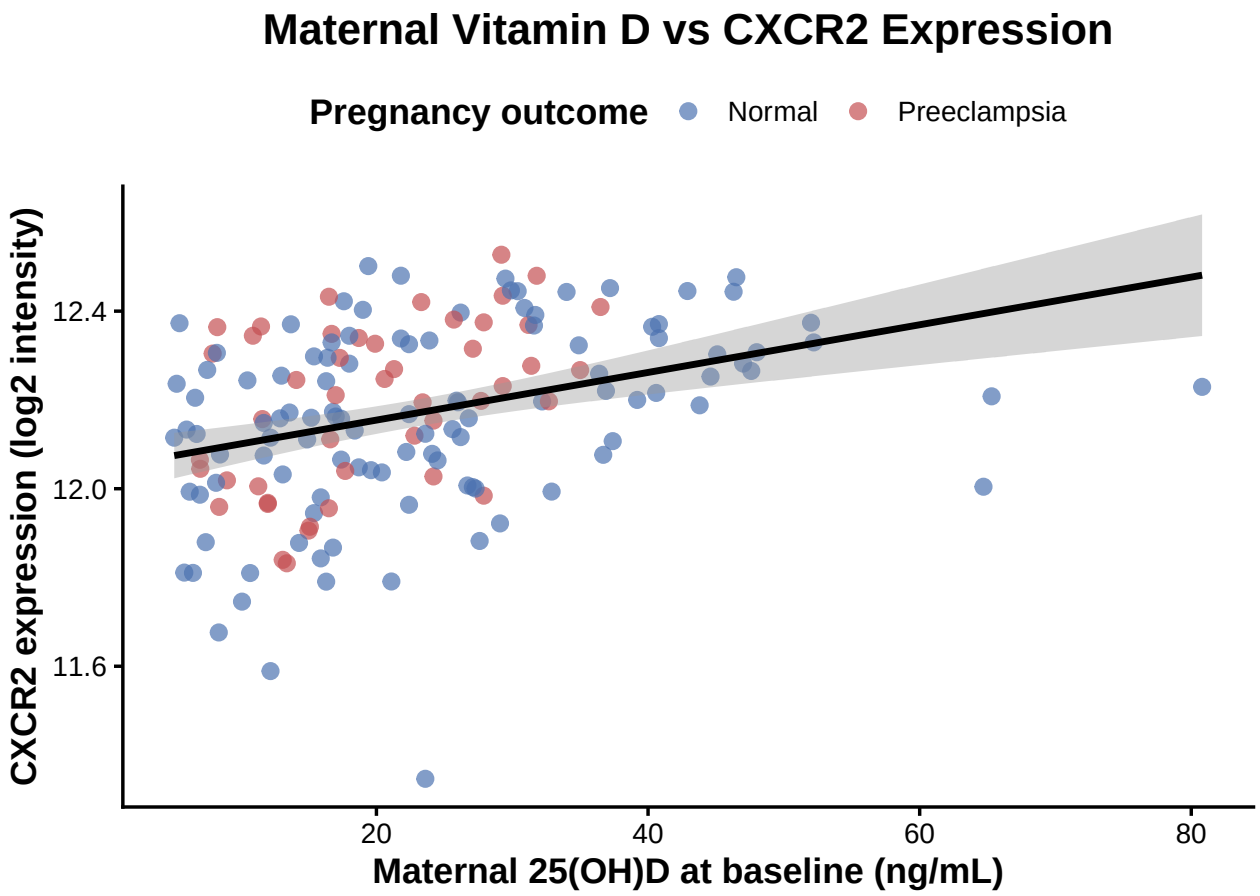
# takes the analysis_long data frame input created from step 3 and 4 as input
# also takes list genes in previous line as input
gene_plots <- plot_genes(analysis_long, genes_of_interest, save = FALSE)
```

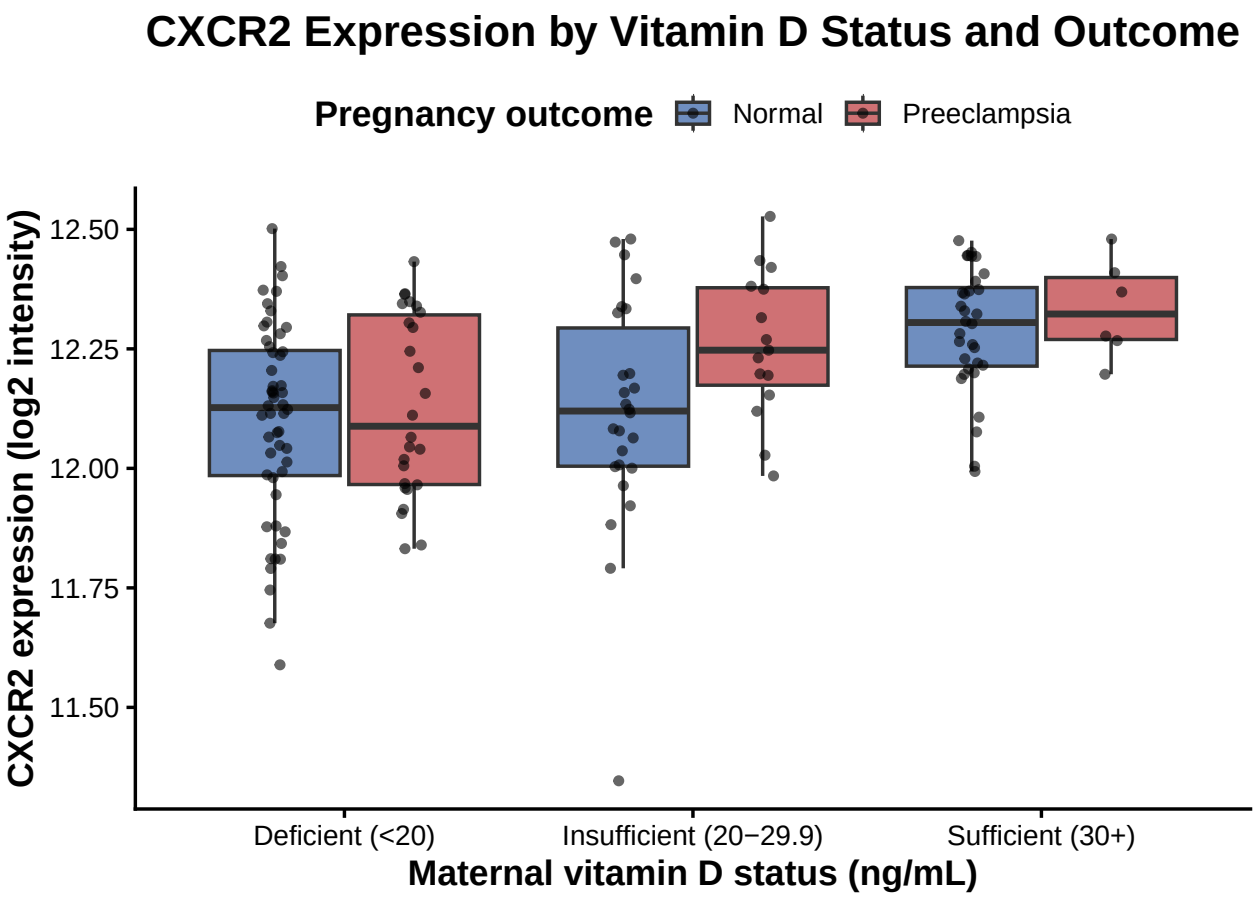

Distribution of PTX3 Expression
Number of samples
0
10
20
4.5
4.8
5.1
5.4
5.7
PTX3 expression (log2 intensity)
Dashed line = median; n = 157

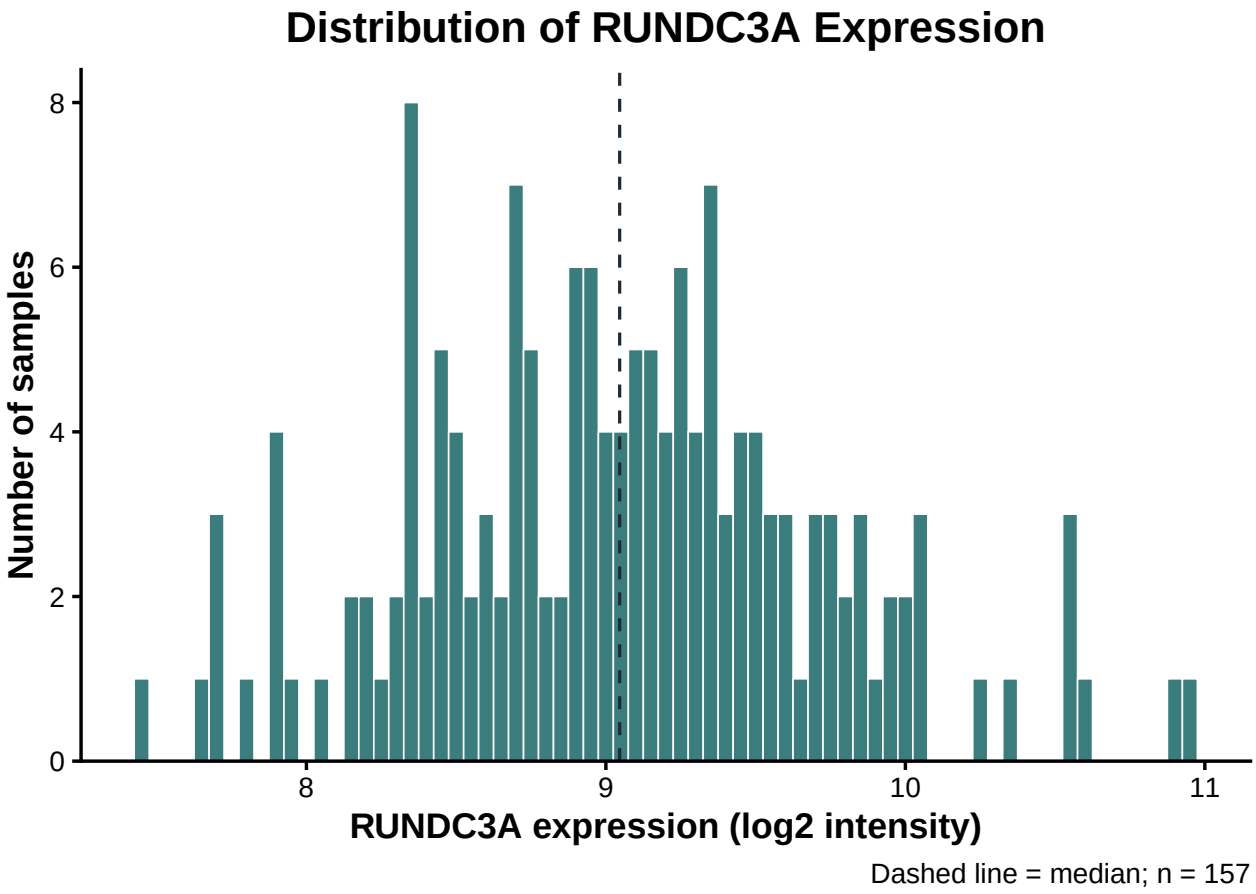

Maternal Vitamin D vs PTX3 Expression
Pregnancy outcome
Normal
Preeclampsia
PTX3 expression (log2 intensity)
5.7
5.4
5.1
4.8
4.5
20
40
60
80
Maternal 25(OH)D at baseline (ng/mL)

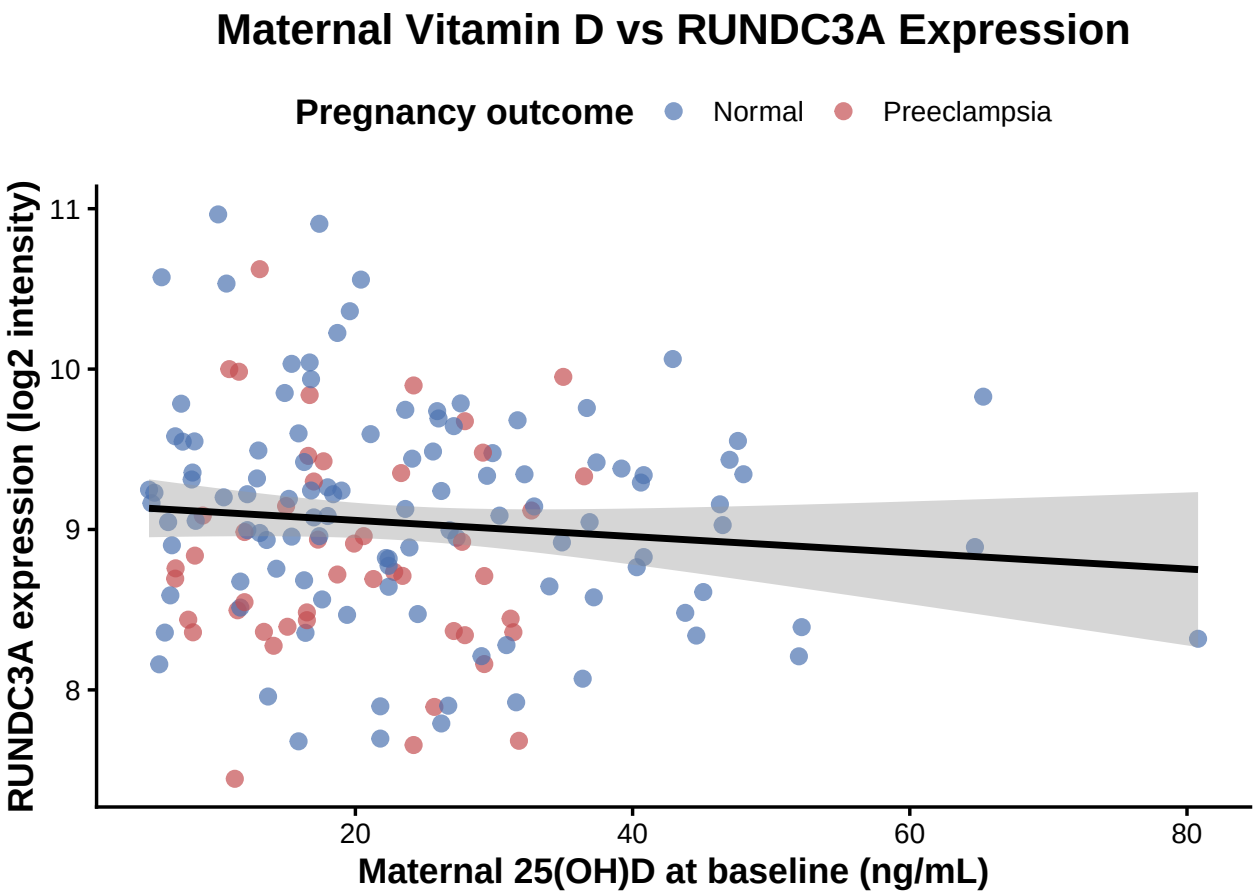

PTX3 Expression by Vitamin D Status and Outcome
Pregnancy outcome
Normal
Preeclampsia
PTX3 expression (log2 intensity)
5.7
5.4
5.1
4.8
4.5
Deficient (<20)
Insufficient (20−29.9)
Sufficient (30+)
Maternal vitamin D status (ng/mL)

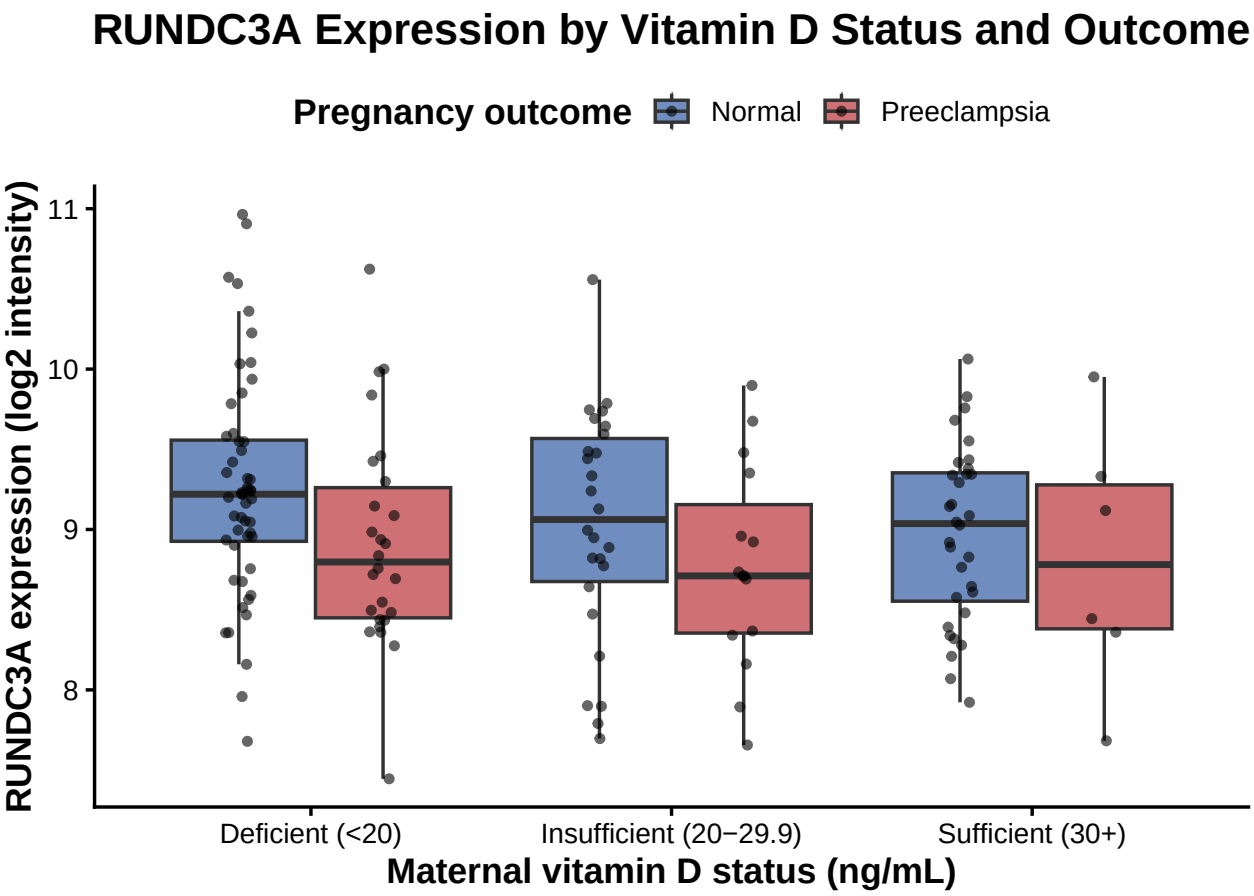

Distribution of CXCR2 Expression
Number of samples
0
5
10
15
20
11.6
12.0
12.4
CXCR2 expression (log2 intensity)
Dashed line = median; n = 157


Maternal Vitamin D vs CXCR2 Expression
Pregnancy outcome
Normal
Preeclampsia
CXCR2 expression (log2 intensity)
12.4
12.0
11.6
20
40
60
80
Maternal 25(OH)D at baseline (ng/mL)


CXCR2 Expression by Vitamin D Status and Outcome
Pregnancy outcome
Normal
Preeclampsia
CXCR2 expression (log2 intensity)
12.50
12.25
12.00
11.75
11.50
Deficient (<20)
Insufficient (20−29.9)
Sufficient (30+)
Maternal vitamin D status (ng/mL)


Distribution of RUNDC3A Expression
Number of samples
0
2
4
6
8
8
9
10
11
RUNDC3A expression (log2 intensity)
Dashed line = median; n = 157


Maternal Vitamin D vs RUNDC3A Expression
Pregnancy outcome
Normal
Preeclampsia
RUNDC3A expression (log2 intensity)
11
10
9
8
20
40
60
80
Maternal 25(OH)D at baseline (ng/mL)



8. Summary statistics for the three genes

Statistics cited in the Results section for each gene: median and SD of expression, Spearman correlation with vitamin D, and p-values for outcome (Wilcoxon) and vitamin D status (Kruskal-Wallis).

```
gene_stats <- analysis_long |>
  filter(GENE %in% genes_of_interest) |>
  group_by(GENE) |>
  summarise(
    median_expr = median(expression),
    sd_expr     = sd(expression),
    rho_vitD    = cor(expression, mat_vitD, method = "spearman"),
    p_vitD      = cor.test(expression, mat_vitD, method = "spearman",
                           exact = FALSE)$p.value,
    p_outcome   = wilcox.test(expression ~ preg_preeclampsia)$p.value,
    p_vitD_cat  = kruskal.test(expression ~ vitD_status)$p.value
  )

gene_stats |> mutate(across(where(is.numeric), ~ signif(.x, 3)))
```

```
## # A tibble: 3 x 7
##   GENE    median_expr sd_expr rho_vitD       p_vitD p_outcome p_vitD_cat
##   <chr>         <dbl>   <dbl>    <dbl>        <dbl>     <dbl>      <dbl>
## 1 CXCR2         12.2    0.204   0.413  0.0000000791    0.422  0.00000609
## 2 PTX3           4.87   0.172  -0.0963 0.23            0.59   0.176
## 3 RUNDC3A        9.05   0.679  -0.0732 0.362           0.0207 0.346
```

9. Summary statistics for the table

Three continuous covariates (vitamin D, maternal age, BMI) shown as mean (SD) and two categorical covariates (pregnancy outcome as the strata, vitamin D status) shown as n (%). P-values: Welch t-test for continuous covariates and chi-square test for vitamin D status.

```
# mean (SD) of each continuous covariate for all women (Overall column)
covar_clean |>
  summarise(across(c(mat_vitD, mat_age_yrs, mat_BMI),
                   list(mean = mean, sd = sd))) |>
  mutate(across(everything(), ~ round(.x, 1)))
```

```
## # A tibble: 1 x 6
##   mat_vitD_mean mat_vitD_sd mat_age_yrs_mean mat_age_yrs_sd mat_BMI_mean
##           <dbl>       <dbl>            <dbl>          <dbl>        <dbl>
## 1            23        13.1             26.4              5         28.9
## # i 1 more variable: mat_BMI_sd <dbl>
```

```
# mean (SD) of each continuous covariate by pregnancy outcome
covar_clean |>
  group_by(preg_preeclampsia) |>
  summarise(across(c(mat_vitD, mat_age_yrs, mat_BMI),
                   list(mean = mean, sd = sd))) |>
  mutate(across(where(is.numeric), ~ round(.x, 1)))
```

```
## # A tibble: 2 x 7
##   preg_preeclampsia mat_vitD_mean mat_vitD_sd mat_age_yrs_mean mat_age_yrs_sd
##   <fct>                    <dbl>       <dbl>            <dbl>          <dbl>
## 1 Normal                    24.4        14.5             26.6            5.1
## 2 Preeclampsia              19.7         8.3             25.8            4.9
## # i 2 more variables: mat_BMI_mean <dbl>, mat_BMI_sd <dbl>
```

```
# p-values comparing normal vs preeclampsia (Welch t-test)
t.test(mat_vitD    ~ preg_preeclampsia, data = covar_clean)$p.value
```

```
## [1] 0.01278723
```

```
t.test(mat_age_yrs ~ preg_preeclampsia, data = covar_clean)$p.value
```

```
## [1] 0.3281601
```

```
t.test(mat_BMI     ~ preg_preeclampsia, data = covar_clean)$p.value
```

```
## [1] 0.02437653
```

```
# n (%) in each vitamin D status for all women (Overall column)
covar_clean |>
  count(vitD_status) |>
  mutate(percent = round(100 * n / sum(n)))
```

```
## # A tibble: 3 x 3
##   vitD_status               n percent
##   <fct>                 <int>   <dbl>
## 1 Deficient (<20)          78      50
## 2 Insufficient (20-29.9)   41      26
## 3 Sufficient (30+)         38      24
```

```
# n (%) in each vitamin D status within each pregnancy outcome
covar_clean |>
```

```
  count(preg_preeclampsia, vitD_status) |>
  group_by(preg_preeclampsia) |>
  mutate(percent = round(100 * n / sum(n)))
```

```
## # A tibble: 6 x 4
## # Groups:   preg_preeclampsia [2]
##   preg_preeclampsia vitD_status               n percent
##   <fct>             <fct>                 <int>   <dbl>
## 1 Normal            Deficient (<20)          52      47
## 2 Normal            Insufficient (20-29.9)   26      24
## 3 Normal            Sufficient (30+)         32      29
## 4 Preeclampsia      Deficient (<20)          26      55
## 5 Preeclampsia      Insufficient (20-29.9)   15      32
## 6 Preeclampsia      Sufficient (30+)          6      13
```

```
# p-value: does vitamin D status differ by outcome? (chi-square test)
chisq.test(table(covar_clean$vitD_status,
                 covar_clean$preg_preeclampsia))$p.value
```

```
## [1] 0.08546922
```

10. Heatmap of 10 genes with tracking bars

The three genes of interest plus the seven most variable remaining genes. Each gene is z-scored across samples so genes with different expression levels share one color scale.

```
# 7 most variable genes, not counting the 3 genes of interest
top_var_genes <- analysis_long |>
  filter(!GENE %in% genes_of_interest) |>
  group_by(GENE) |>
  summarise(variance = var(expression)) |>
  arrange(desc(variance)) |>
  slice_head(n = 7) |>
  pull(GENE)

heatmap_genes <- c(genes_of_interest, top_var_genes)
```

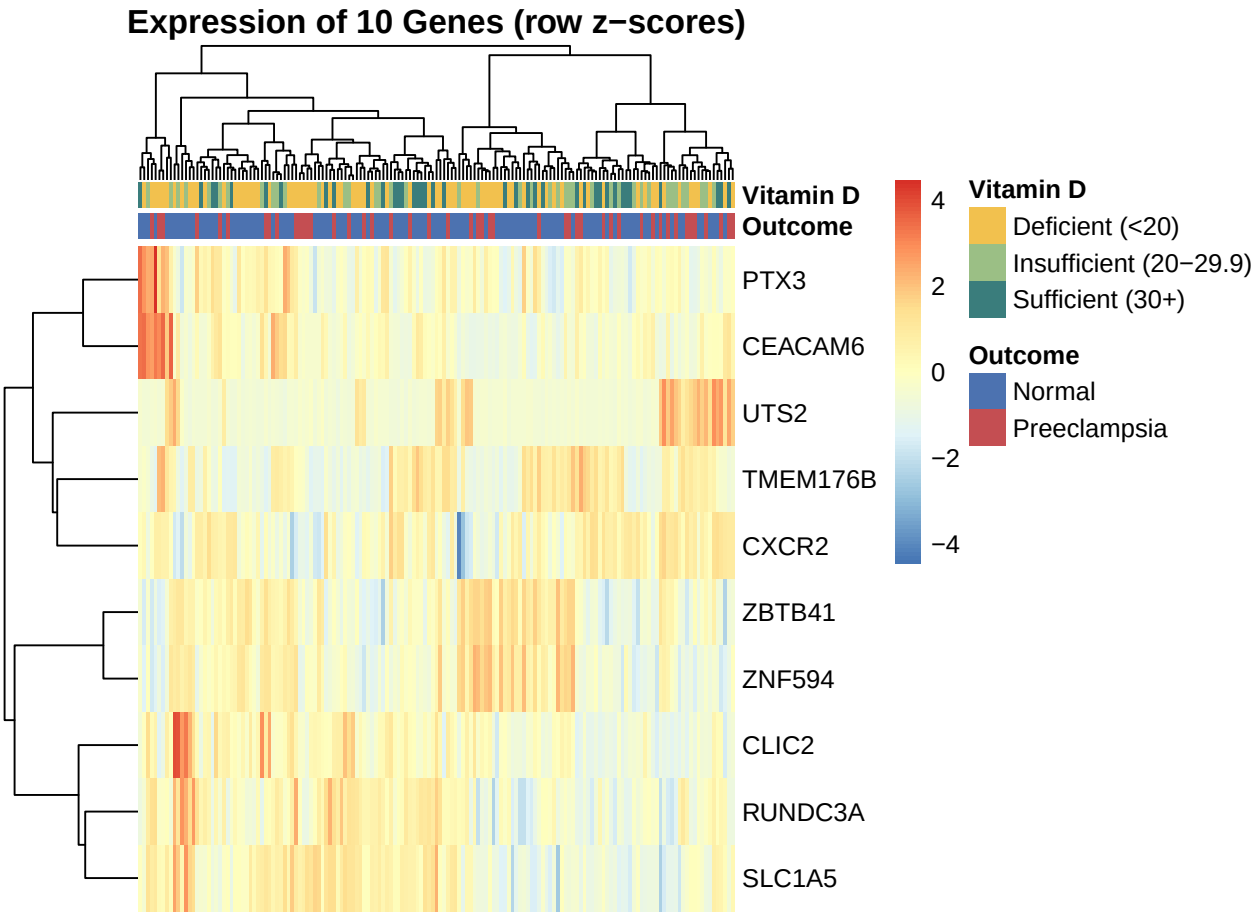
```
# matrix with genes as rows and samples as columns
heat_mat <- analysis_long |>
  filter(GENE %in% heatmap_genes) |>
  select(GENE, geoID, expression) |>
  pivot_wider(names_from = geoID, values_from = expression) |>
  column_to_rownames("GENE") |>
  as.matrix()

# tracking bars: one row per sample, row names = sample IDs
tracking <- covar_clean |>
  select(geoID, Outcome = preg_preeclampsia, `Vitamin D` = vitD_status) |>
  column_to_rownames("geoID")

# colors for the tracking bars
tracking_colors <- list(
  Outcome     = outcome_colors,
  `Vitamin D` = c("Deficient (<20)"        = "#F2C14E",
                  "Insufficient (20-29.9)" = "#9BBF85",
```

```
                  "Sufficient (30+)"       = "#3A7D7C"))

heat <- pheatmap(heat_mat,
                 scale = "row", # z-score each gene
                 clustering_method = "ward.D2",  # cluster genes and samples
                 annotation_col = tracking, # tracking bars
                 annotation_colors = tracking_colors,
                 show_colnames = FALSE, # 157 sample IDs unreadable
                 main = "Expression of 10 Genes (row z-scores)")
```




```
# save a copy
ggsave("figures/heatmap_10genes.png", heat, width = 9, height = 5.5, dpi = 300)
```

11. Additional plot: treemap of group sizes and expression

A treemap splits a rectangle into tiles whose area is proportional to a count. Here each tile is one group of women (pregnancy outcome x vitamin D status), so the tile area shows how many women are in that group. Tile color shows how that group's median expression compares with the gene's overall median (red = higher, blue = lower).

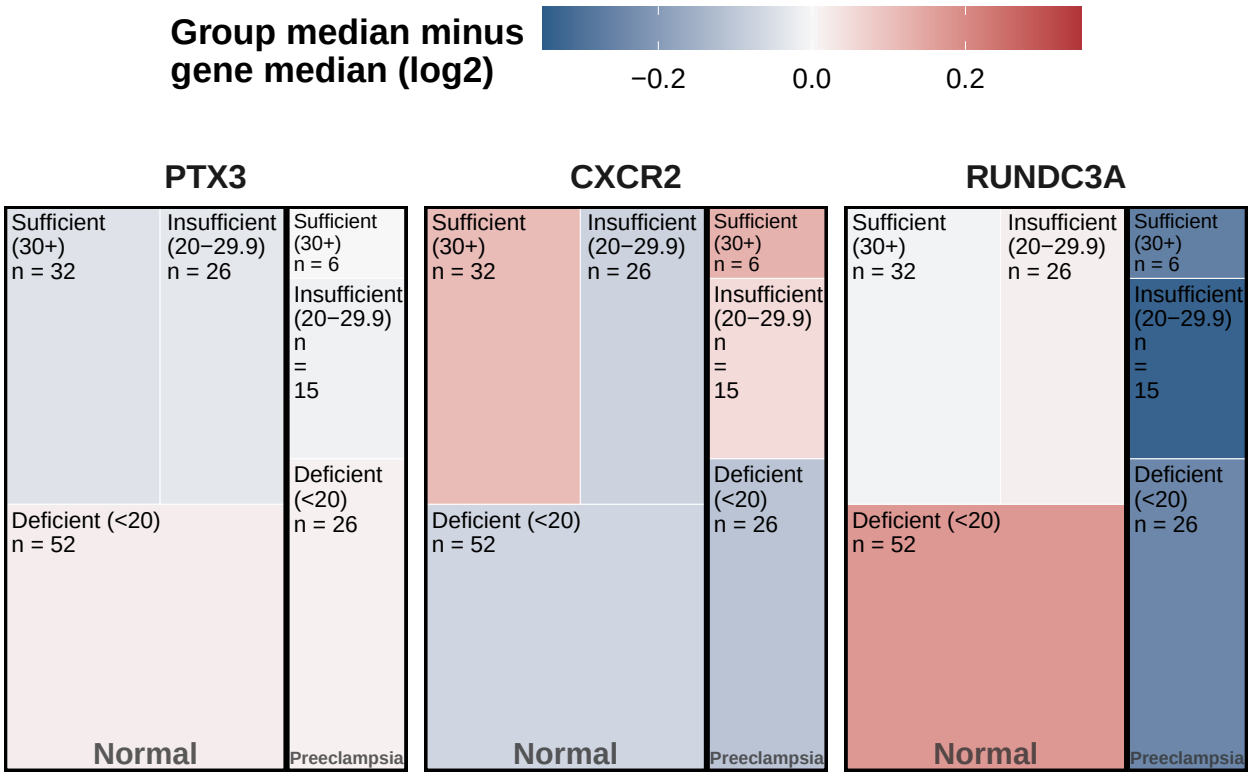
```
library(treemapify)

treemap_df <- analysis_long |>
  filter(GENE %in% genes_of_interest) |>
  # expression relative to each gene's overall median, so all 3 genes
```

```
  # can share one color scale
  group_by(GENE) |>
  mutate(rel_expr = expression - median(expression)) |>
  # one row per gene x outcome x vitamin D status group
  group_by(GENE, preg_preeclampsia, vitD_status) |>
  summarise(n = n(), median_rel = median(rel_expr), .groups = "drop") |>
  mutate(GENE  = factor(GENE, levels = genes_of_interest), # panel order
         label = paste0(vitD_status, "\nn = ", n))  # tile label
```

```
p_treemap <- ggplot(treemap_df, aes(area = n, fill = median_rel, label = label,
                                    subgroup = preg_preeclampsia)) +
  geom_treemap(color = "white") + # one tile per group
  geom_treemap_subgroup_border(color = "black", size = 3) + # outline each outcome
  geom_treemap_subgroup_text(place = "bottom", size = 12, # outcome label
                             fontface = "bold", alpha = 0.8) +
  geom_treemap_text(place = "topleft", size = 9, reflow = TRUE) + # group label
  # blue = lower, white = same, red = higher than the gene's median
  scale_fill_gradient2(low = "#2C5D8A", mid = "#F7F7F7", high = "#B2343A",
                       midpoint = 0, limits = c(-0.35, 0.35),
                       name = "Group median minus\ngene median (log2)") +
  facet_wrap(~ GENE) + # one panel per gene
  labs(title = "Group Sizes and Relative Expression by Outcome and Vitamin D Status",
       caption = paste("Tile area is the number of women;",
                       "color is group median minus the gene's overall median")) +
  theme_pub +
  theme(strip.text = element_text(face = "bold", size = 13),
        strip.background = element_blank(),
        plot.title = element_text(size = 14),
        legend.key.width = unit(1.5, "cm"))

p_treemap
```



Tile area is the number of women; color is group median minus the gene's overall median

```
# save the figure
ggsave("figures/treemap_groups.png", p_treemap, width = 11, height = 5, dpi = 300)
```

AI use statement: Claude (Anthropic) was used to help with str_removal for initial data cleaning and some GitHub debugging issues for function creation. Also consulted Claude for help specifically with including the ggsave portion in the function writing, and consulted google as well for ggsave arguments within the function. Most of the code remained the same as submission 1 and 2. Also, consulted Claude (Anthropic) for help in the treemap, and the annotations, in order to have annotations on each tile.